

```
# =====
# TASK 1: DATA PREPROCESSING & THREE-WAY SPLIT SYSTEM (TRAIN/VAL/TEST)
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, PredefinedSplit, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Load the dataset (using your verified 'ReadMe.csv' name)
df = pd.read_csv('ReadMe.csv')

# 2. Impute missing arrival delays with the column median
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].median())

# 3. Binary map the target feature
df['satisfaction'] = df['satisfaction'].map({'satisfied': 1, 'dissatisfied': 0})

# 4. Cleanly convert remaining categorical features via dummy variables
categorical_cols = ['Customer Type', 'Type of Travel', 'Class']
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

X = df_encoded.drop(columns=['satisfaction'])
y = df_encoded['satisfaction']

# 5. Execute 3-Way Split: 60% Train, 20% Validation, 20% Test
# First, extract 20% held-out test data
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# Second, split remaining 80% into Train (60% total) and Validation (20% total)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42)

print(f"✅ Three-Way Data Split Executed Successfully:")
print(f"    - Training Set    : {X_train.shape[0]} samples (60%)")
print(f"    - Validation Set  : {X_val.shape[0]} samples (20%)")
print(f"    - Testing Set     : {X_test.shape[0]} samples (20%)")
```

```
✅ Three-Way Data Split Executed Successfully:
    - Training Set    : 77928 samples (60%)
    - Validation Set  : 25976 samples (20%)
    - Testing Set     : 25976 samples (20%)
```

```
# =====
# TASK 2: GRIDSEARCHCV SETUP USING PREDEFINEDSPLIT
# =====

# 1. Concatenate Training and Validation data into a single search space matrix
X_combined = pd.concat([X_train, X_val]).reset_index(drop=True)
y_combined = pd.concat([y_train, y_val]).reset_index(drop=True)

# 2. Map split array indicators: -1 for training rows, 0 for validation rows
# This ensures GridSearchCV trains ONLY on Train indices and validates ONLY on
```

```

split_index = [-1] * len(X_train) + [0] * len(X_val)
pds = PredefinedSplit(test_fold=split_index)

# 3. Define hyperparameter tuning grid bounds
param_grid = {
    'max_depth': [10, 12, 15],
    'n_estimators': [50, 100],
    'min_samples_leaf': [2, 5, 10]
}

# 4. Initialize search pipeline targeting F1 optimization
rf_base = RandomForestClassifier(random_state=42, n_jobs=-1)
grid_search = GridSearchCV(estimator=rf_base, param_grid=param_grid, cv=pds, scoring='f1')
grid_search.fit(X_combined, y_combined)

print("\n🌟 Optimal Random Forest Hyperparameters Found:")
print(grid_search.best_params_)

# Isolate the optimal model
best_rf_model = grid_search.best_estimator_

```

Fitting 1 folds for each of 18 candidates, totalling 18 fits

🌟 Optimal Random Forest Hyperparameters Found:
{'max_depth': 15, 'min_samples_leaf': 2, 'n_estimators': 100}

```

# =====
# TASK 3: METRIC ANALYSIS ON HELD-OUT TEST DATA
# =====

# 1. Evaluate the optimized Random Forest model on the test set
y_pred_rf = best_rf_model.predict(X_test)

rf_acc = accuracy_score(y_test, y_pred_rf)
rf_prec = precision_score(y_test, y_pred_rf)
rf_rec = recall_score(y_test, y_pred_rf)
rf_f1 = f1_score(y_test, y_pred_rf)

print("--- 🎯 Random Forest Test Set Performance Metrics ---")
print(f"Accuracy : {rf_acc*100:.2f}%")
print(f"Precision : {rf_prec*100:.2f}%")
print(f"Recall : {rf_rec*100:.2f}%")
print(f"F1-Score : {rf_f1*100:.2f}%\n")

print("Classification Summary:\n", classification_report(y_test, y_pred_rf))

```

```

--- 🎯 Random Forest Test Set Performance Metrics ---
Accuracy : 94.72%
Precision : 95.68%
Recall : 94.62%
F1-Score : 95.15%

```

```

Classification Summary:
              precision    recall  f1-score   support

0             0.94         0.95         0.94        11759
1             0.96         0.95         0.95        14217

```

accuracy			0.95	25976
macro avg	0.95	0.95	0.95	25976
weighted avg	0.95	0.95	0.95	25976

```
# =====
# TASK 4: SINGLE DECISION TREE BENCHMARK INTERPRETATION
# =====

# Train an identical decision tree configuration to measure model performance
dt_baseline = DecisionTreeClassifier(max_depth=10, random_state=42)
dt_baseline.fit(X_train, y_train)
y_pred_dt = dt_baseline.predict(X_test)

dt_acc = accuracy_score(y_test, y_pred_dt)
dt_prec = precision_score(y_test, y_pred_dt)
dt_rec = recall_score(y_test, y_pred_dt)
dt_f1 = f1_score(y_test, y_pred_dt)

print("--- 📊 Side-by-Side Performance Delta ---")
print(f"Accuracy Improvement : {((rf_acc - dt_acc)*100):+.2f}%")
print(f"F1-Score Improvement : {((rf_f1 - dt_f1)*100):+.2f}%")
```

```
--- 📊 Side-by-Side Performance Delta ---
Accuracy Improvement : +2.50%
F1-Score Improvement : +2.30%
```

```
# =====
# TASK 5: ENSEMBLE DRIVER INSIGHT PLOTTING
# =====

# Extract feature relative importance metrics
importances = best_rf_model.feature_importances_
feature_ranking = pd.DataFrame({
    'Operational Service Driver': X.columns,
    'Importance Weight': importances
}).sort_values(by='Importance Weight', ascending=False).reset_index(drop=True)

print("\n--- 🛒 Top 5 Customer Satisfaction Drivers ---")
print(feature_ranking.head(5))

# Plot top features
plt.figure(figsize=(10, 5))
sns.barplot(data=feature_ranking.head(8), x='Importance Weight', y='Operational Service Driver')
plt.title('Top 8 Operational Performance Drivers of Airline Passenger Satisfaction')
plt.xlabel('Ensemble Feature Importance Weight')
plt.tight_layout()
plt.show()
```

```

--- 🛒 Top 5 Customer Satisfaction Drivers ---
Operational Service Driver Importance Weight
0 Inflight entertainment 0.263067
1 Seat comfort 0.150071
2 Online support 0.076209
3 Ease of Online booking 0.075803
4 Food and drink 0.046037
/tmp/ipykernel_1880/495509445.py:17: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version. Use `color` instead.

```

sns.barplot(data=feature_ranking.head(8), x='Importance Weight', y='Operational Service Driver')

```

